Experiment 18: RNN, LSTM and GRU Hyperparameter Experimentation

Question

Problem Statement: A product team classifies customer reviews as positive or negative. It must pick the best recurrent model and its settings within a fixed compute budget. The IMDB movie-review sentiment dataset (or product reviews from an e-commerce site). The objective is to tune RNN, LSTM and GRU models automatically with KerasTuner. Use TensorFlow, Keras, NumPy, KerasTuner, Matplotlib.

Tasks:

1. Tokenise and pad the reviews; add an embedding layer.
2. Define a model-builder function that chooses the cell type (RNN, LSTM, GRU).
3. Tune units, layers, dropout, learning rate and embedding size with KerasTuner.
4. Report the best configuration for each cell type.
5. Compare accuracy, parameter count and training time.

Code

```
"""Experiment 18: RNN, LSTM and GRU Hyperparameter Experimentation with KerasTuner (IMDB)"""
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import keras_tuner as kt                      # pip install keras-tuner

np.random.seed(42); tf.random.set_seed(42)
USE_SYNTHETIC = False        # keep False; True only generates random data to smoke-test the code
NUM_WORDS, MAXLEN = 10000, 200
N_TRAIN, N_TEST = 8000, 4000                  # subsets so that the search fits a small compute budget
MAX_TRIALS, SEARCH_EPOCHS, FINAL_EPOCHS = 4, 3, 4

# ---------------- Task 1: tokenise (IMDB is pre-tokenised), pad, embedding later ----------------
if USE_SYNTHETIC:
    x_train = np.random.randint(1, NUM_WORDS, (600, MAXLEN)); y_train = np.random.randint(0, 2, 600)
    x_test = np.random.randint(1, NUM_WORDS, (300, MAXLEN)); y_test = np.random.randint(0, 2, 300)
else:
    (x_train, y_train), (x_test, y_test) = keras.datasets.imdb.load_data(num_words=NUM_WORDS)
    x_train = keras.utils.pad_sequences(x_train[:N_TRAIN], maxlen=MAXLEN); y_train = y_train[:N_TRAIN]
    x_test = keras.utils.pad_sequences(x_test[:N_TEST], maxlen=MAXLEN); y_test = y_test[:N_TEST]
print("Train:", x_train.shape, "Test:", x_test.shape)

CELLS = {"RNN": layers.SimpleRNN, "LSTM": layers.LSTM, "GRU": layers.GRU}

# ---------------- Tasks 2 & 3: model builder with tunable hyperparameters ----------------
def make_builder(cell_name):
    def build(hp):
        cell_name_hp = hp.Choice("cell", [cell_name])                    # cell type is a hyperparameter
        cell = CELLS[cell_name_hp]
        units = hp.Int("units", 32, 128, step=32)
        n_layers = hp.Int("rnn_layers", 1, 2)
        dropout = hp.Float("dropout", 0.0, 0.5, step=0.1)
        lr = hp.Float("lr", 1e-4, 1e-2, sampling="log")
        emb = hp.Choice("embedding_dim", [32, 64, 128])
        m = keras.Sequential([layers.Input((MAXLEN,)), layers.Embedding(NUM_WORDS, emb)])
        for i in range(n_layers):
            m.add(cell(units, return_sequences=(i < n_layers - 1)))
            m.add(layers.Dropout(dropout))
        m.add(layers.Dense(1, activation="sigmoid"))
        m.compile(optimizer=keras.optimizers.Adam(lr), loss="binary_crossentropy", metrics=["accuracy"])
        return m
    return build

# ---------------- Tasks 3-5: search each cell type, retrain best, compare ----------------
rows, best_configs = [], {}
for name in CELLS:
```

```
    tuner = kt.RandomSearch(make_builder(name), objective="val_accuracy", max_trials=MAX_TRIALS,
                            directory="kt_logs", project_name=f"imdb_{name}", overwrite=True, seed=42)
    t0 = time.time()
    tuner.search(x_train, y_train, validation_split=0.2, epochs=SEARCH_EPOCHS, batch_size=64, verbose=0)
    search_time = time.time() - t0

    best_hp = tuner.get_best_hyperparameters(1)[0]
    best_configs[name] = {k: best_hp.get(k) for k in ["units", "rnn_layers", "dropout", "lr",
"embedding_dim"]}
    print(f"\n[{name}] best hyperparameters: {best_configs[name]}")

    model = tuner.hypermodel.build(best_hp)                                # retrain best config
    t0 = time.time()
    model.fit(x_train, y_train, validation_split=0.2, epochs=FINAL_EPOCHS, batch_size=64, verbose=0)
    train_time = time.time() - t0
    test_acc = model.evaluate(x_test, y_test, verbose=0)[1]
    rows.append({"Cell": name, **best_configs[name], "Params": model.count_params(),
                 "Search time (s)": round(search_time, 1), "Final train time (s)": round(train_time, 1),
                 "Test accuracy": round(test_acc, 4)})
    print(f"[{name}] test accuracy = {test_acc:.4f}, params = {model.count_params():,}, train time =
{train_time:.1f}s")

table = pd.DataFrame(rows).set_index("Cell")
pd.set_option("display.width", 220); pd.set_option("display.max_columns", None)
print("\n", table)
table.to_csv("kerastuner_results.csv")

fig, ax = plt.subplots(1, 3, figsize=(15, 4))
ax[0].bar(table.index, table["Test accuracy"]); ax[0].set_title("Test accuracy"); ax[0].set_ylim(0.4, 1.0)
ax[1].bar(table.index, table["Params"]); ax[1].set_title("Parameter count")
ax[2].bar(table.index, table["Final train time (s)"]); ax[2].set_title("Training time (s)")
plt.tight_layout(); plt.show()
```

Output

